Global E-Commerce Architecture Interview Guide

Portfolio Project 2 • Version 1.0

Use this guide to explain your design decisions clearly and honestly; do not memorize it word for word.

1. Your 60-second architecture pitch

I would approach this as a phased modernization rather than a wholesale rewrite. First, establish a governed AWS landing zone and baseline current cost, latency, availability, and dependencies. Then migrate workloads in controlled waves, use CloudFront for eligible cacheable content, run suitable containerized services on ECS Fargate, and select relational or NoSQL stores based on access patterns and consistency requirements. Use SQS only for workflows that can safely complete asynchronously, with idempotent consumers and dead-letter handling. Define workload-specific RTO/RPO, implement multi-AZ patterns, and choose a recovery-region strategy based on business impact and tested recovery needs. Finally, validate targets through load tests, recovery exercises, security reviews, and a fully costed model. The targets remain hypotheses until measured.

2. Five decisions to defend

Phased migration over big-bang: lowers cutover risk and enables learning from pilot waves. Trade-off: temporary dual-running cost and longer coexistence.

CloudFront for eligible content: can improve global delivery and reduce origin load. Trade-off: cache-key design and the risk of exposing personalized data if rules are wrong. Default: do not cache authenticated or personalized responses.

ECS Fargate for selected services: reduces host-management overhead. Trade-off: may not be cheapest at steady high utilization; scaling, deployment, networking, and observability still require design.

Database by workload: relational databases suit SQL, constraints, and transactional needs; DynamoDB suits defined key-based access patterns at scale. Trade-off: multiple data models add operational complexity.

Primary plus recovery region before active-active: often easier to operate if business RTO/RPO permit it. Trade-off: recovery can take longer and must be tested. Active-active needs a business case.

3. Common interview questions

How would you prove 60% cost reduction? Establish a tagged baseline by workload and service. Model rightsizing, autoscaling, non-production scheduling, commitments for stable usage, storage lifecycle, logging retention, eligible edge caching, database optimization, and NAT/VPC-endpoint economics. Include data transfer, cross-region replication, support, and operational effort. Compare forecasts with actual bills after each wave.

How would you achieve latency below 150 ms globally? Define the user journeys, regions, percentile, and measurement points. Separate static delivery from dynamic API latency. Use CloudFront for eligible content, measure origin latency, inspect database and dependency time, and test under representative peak traffic. Consider regional app placement or Global Accelerator only after measuring the bottleneck.

Why not active-active in three regions? It adds complexity around data ownership, writes, replication lag, conflict handling, routing, incident operations, and cost. Start with business RTO/RPO and consistency needs. If active-active is required, define data ownership and conflict strategy per domain and prove it through failure testing.

Why both PostgreSQL and DynamoDB? Only if distinct workloads justify them. PostgreSQL suits relational transactions and SQL queries; DynamoDB suits access patterns designed around keys and predictable scale. Polyglot persistence adds operational burden, so each store needs a clear workload-level reason.

What must SQS consumers do? Design for at-least-once delivery and possible duplicate processing. Consumers need idempotency, durable state transitions, retry/backoff, visibility-timeout alignment, DLQ alarms, and replay procedures. Queue features do not replace business-level duplicate protection.

Multi-AZ versus multi-region recovery? Multi-AZ helps tolerate certain availability-zone failures within a region. Multi-region recovery addresses a broader regional outage and introduces replication, failover, consistency, routing, and cost decisions. Neither proves an RTO/RPO until tested.

What if a migration wave fails? Define success metrics, change window, rollback trigger, data checks, traffic-shift method, owners, and communication plan before cutover. Keep the previous path available until validation passes; monitor after cutover

and conduct a blameless review.

4. Metrics to name

User experience: p50/p95/p99 latency, error rate, order completion, regional synthetic tests.

Application: request rate, task CPU/memory, healthy target count, restart rate, deployment failure rate, dependency latency.

Data: connection saturation, query latency, storage/IO pressure, replication lag, failover/reconnect behavior, backup-restore validation.

Messaging: queue depth, oldest-message age, retry count, DLQ count, consumer throughput and processing latency.

Cost: cost by workload/environment, cost per order where measurable, transfer spend, NAT spend, idle resource cost, budget variance.

Recovery: measured RTO/RPO, data integrity after recovery, runbook completion time, failover/failback results.

5. Gaps to acknowledge honestly

The case study does not yet provide application inventory, actual traffic distribution, data volumes, per-market residency rules, approved RTO/RPO, detailed cost baseline, or benchmark results. These are discovery and validation tasks—not facts to invent.

The proposed AWS Regions are placeholders for the scenario and must be checked against user geography, legal requirements, AWS service availability, latency, and cost.

The 60% cost reduction, sub-150 ms latency, and 99.99% availability are target objectives, not achieved results.

Do not claim to have deployed the architecture or served TechVenture Retail Inc. This is a hypothetical portfolio design.

6. Reusable answer structure

For almost any architecture question: (1) clarify the business requirement, (2) state assumptions, (3) propose the design, (4) explain the trade-off, (5) identify the main failure mode, (6) define the metric/test that validates it, and (7) explain the fallback or rollback.